

1. Protocolo

Dirección de archivo: \src\main\java\rational\

Tecnológico Nacional de México

Instituto Tecnológico de León, Campus 1

Carrera: Ingeniería en Sistemas Computacionales

Materia: Estructura de Datos (Grupo B)

Horario: Lunes y miércoles de 8:45 a 10:25 hrs. y
Viernes de 8:45 a 9:35 hrs.

Alumno: Marlene Inés Moreno Velázquez

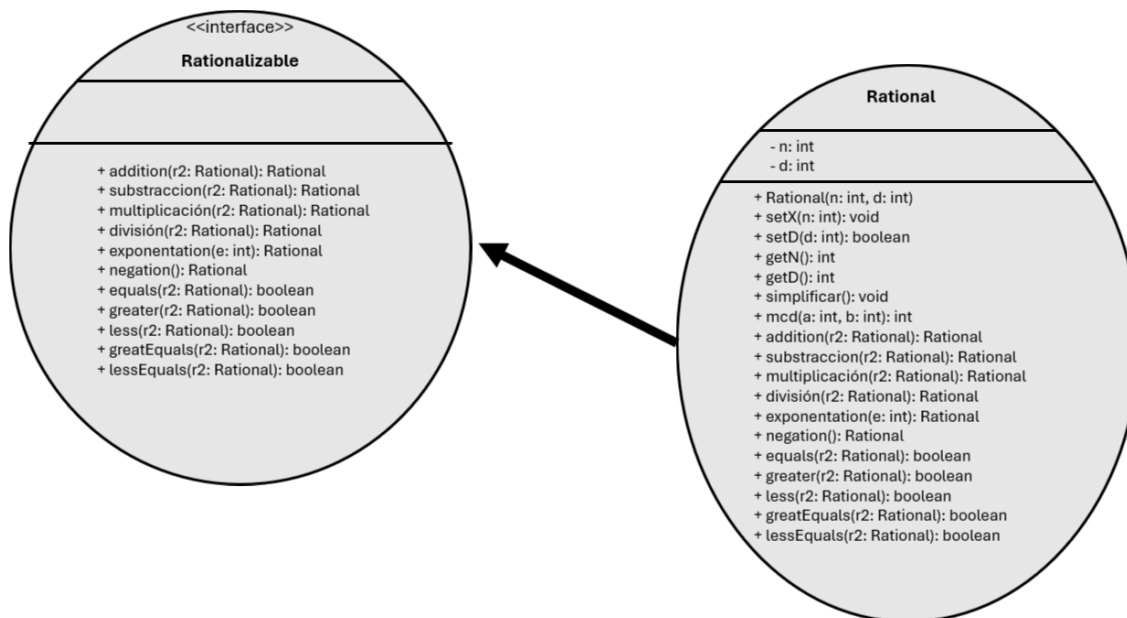
Número de lista: 12

Fecha: 4 de marzo de 2026.



Ejercicio #13: Rational Java

2. Diagrama UML



3. Clase encapsulada Rational

```
package rational;

public class Rational implements Rationalizable {
    private int n;
    private int d;

    public Rational(int n, int d){
        if(d == 0)
            throw new IllegalArgumentException("El denominador no puede ser 0");
        this.n = n;
        this.d = d;
        simplificar();
    }

    public void setX(int n){
        this.n = n;
    }

    public Boolean setD(int d){
        if(d == 0) return false;
        this.d = d;
        return true;
    }

    public int getN(){
        return n;
    }

    public int getD(){
        return d;
    }

    private void simplificar(){
        int mcd = mcd(n,d);
        n = n/mcd;
        d = d/mcd;

        if(d < 0){
            n = -n;
            d = -d;
        }
    }

    private int mcd(int a, int b){
        a = Math.abs(a);
        b = Math.abs(b);

        while(b != 0){
```

```

        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// ===== OPERACIONES =====
public Rational addition(Rational r2){
    int num = n*r2.d + r2.n*d;
    int den = d*r2.d;

    return new Rational(num,den);
}

public Rational addition(int num){
    int newNum = n + num*d;

    return new Rational(newNum,d);
}

public Rational substraccion(Rational r2){
    int num = n*r2.d - r2.n*d;
    int den = d*r2.d;

    return new Rational(num,den);
}

public Rational substraccion(int num){
    int newNum = n - num*d;

    return new Rational(newNum,d);
}

public Rational multiplicacion(Rational r2){
    int num = n*r2.n;
    int den = d*r2.d;

    return new Rational(num,den);
}

public Rational multiplicacion(int num){
    return new Rational(n*num,d);
}

public Rational division(Rational r2){
    int num = n*r2.d;
    int den = d*r2.n;

    return new Rational(num,den);
}

```

```

    }

    public Rational division(int num){
        return new Rational(n,d*num);
    }

    public Rational exponentiation(int e){
        int num = (int)Math.pow(n,e);
        int den = (int)Math.pow(d,e);

        return new Rational(num,den);
    }

    public Rational negation(){
        return new Rational(-n,d);
    }

    // ===== COMPARACIONES =====
    public boolean equals(Rational r2){
        return n*r2.d == r2.n*d;
    }

    public boolean greater(Rational r2){
        return n*r2.d > r2.n*d;
    }

    public boolean less(Rational r2){
        return n*r2.d < r2.n*d;
    }

    public boolean greatEquals(Rational r2){
        return n*r2.d >= r2.n*d;
    }

    public boolean lessEquals(Rational r2){
        return n*r2.d <= r2.n*d;
    }

    public String toString(){
        return n + "/" + d;
    }
}

```

4. Algoritmo

1. Meta: Codificar un programa que permita probar las prestaciones del ADT Rational realizando operaciones entre fracciones como suma, resta, multiplicación, división, exponenciación, negación y comparación.

2. Data:

```
entero numerador1, numerador2, denominador1, denominador2;
Rational fraccion1, fraccion2;
esValido = falso;
terminar mientras (!esValido) empezar
    esValido = llamar a fracc1();
terminar mientras
esValido = falso;
terminar mientras (!esValido) empezar
    esValido = llamar a fracc2();
terminar mientras
fraccion1 = new Rational(numerador1, denominador1);
fraccion2 = new Rational(numerador2, denominador2);
```

2.1 fracc1():

```
esValido = falso; esInd = true;
terminar mientras (!esValido OR esInd) empezar
    Escribir("Ingrese el numerador de la fraccion 1 >>>");
    numerador1 = ?;
    Escribir("Ingrese el denominador de la fraccion 1 >>>");
    denominador1 = ?;
    si (denominador1 != 0) empezar
        esInd = falso;
        si (numerador1 != 0) empezar
            esValido = verdadero;
        de lo contrario empezar
            Escribir("Fraccion equivalente a 0.");
        terminar
    de lo contrario empezar
        Escribir("El denominador no puede ser 0.");
    terminar si
terminar mientras
retornar verdadero
```

2.2 fracc2():

```
esValido = falso; esInd = true;
terminar mientras (!esValido OR esInd) empezar
    Escribir("Ingrese el numerador de la fraccion 2 >>>");
    numerador2 = ?;
    Escribir("Ingrese el denominador de la fraccion 2 >>>");
    denominador2 = ?;
    si (denominador2 != 0) empezar
        esInd = falso;
        si (numerador2 != 0) empezar
            esValido = verdadero;
        de lo contrario empezar
```

```

        Escribir("Fraccion equivalente a 0.");
    terminar
de lo contrario empezar
    Escribir("El denominador no puede ser 0.");
    terminar si
terminar mientras

retornar verdadero

```

3. Procesos:

```

Escribir("Ingrese el inciso de la operacion que desea hacer:")
+ "\n\t1. Sumar"
+ "\n\t2. Restar"
+ "\n\t3. Multiplicar"
+ "\n\t4. Dividir"
+ "\n\t5. Exponente"
+ "\n\t6. Negacion"
+ "\n\t7. Comparar"
+ "\nSu eleccion >>>");

```

```
opcion = ?; Rational resultado;
```

```

SEGUN (opcion) HACER
    caso 1: resultado = fraccion1.addition(fraccion2);
    caso 2: resultado = fraccion1.substraccion(fraccion2);
    caso 3: resultado = fraccion1.multiplicacion(fraccion2);
    caso 4: resultado = fraccion1.division(fraccion2);
    caso 5: e = 0; Escribir("Ingrese el exponente >>>"); e = ?;
        resultado = fraccion1.exponentation(e);
    caso 6: resultado = fraccion1.negation();
    caso 7: eq = fraccion1.equals(fraccion2);
FIN SEGUN

```

4. Resultados:

```

SEGUN (opcion) HACER
    caso 1: Escribir(fraccion1.toString + " + " + fraccion2.toString
        + " = " + resultado.toString);
    caso 2: Escribir(fraccion1.toString + " - " + fraccion2.toString
        + " = " + resultado.toString);
    caso 3: Escribir(fraccion1.toString + " x " + fraccion2.toString
        + " = " + resultado.toString);
    caso 4: Escribir(fraccion1.toString + " ÷ " + fraccion2.toString
        + " = " + resultado.toString);
    caso 5: Escribir(fraccion1.toString + " ^ " + e
        + " = " + resultado.toString);
    caso 6: Escribir(" Negacion de " + fraccion1.toString
        + " = " + resultado.toString);
    caso 7: Escribir(fraccion1.toString + " es igual a "+fraccion2.toString

```

FIN SEGUN

5. Navegabilidad main():

```
llamar a meta();
llamar a data();
llamar a proceso();
llamar a resultados();
```

5. Algoritmo codificado en Java

[illegible]

```

        if (n1 != 0) {
            isValid = true;
            if (d1 != 0) isInd = false;
            else entry.outError("El denominador no puede ser 0.",
                                "INDETERMINACION");
        } else
            entry.outError("Si el numerador es 0, toda la fraccion es "
                            +"igual a 0.", "FRACCION CON VALOR NULO");
    }
    return true;
}

boolean fracc2() {
    boolean isValid = false, isInd = true;

    while (!isValid || isInd) {
        n2 = entry.entryInt(0, "Fraccion 2", "Ingrese numerador");
        d2 = entry.entryInt(0, "Fraccion 2", "Ingrese denominador");
        if (n2 != 0) {
            isValid = true;
            if (d2 != 0) isInd = false;
            else entry.outError("El denominador no puede ser 0.",
                                "INDETERMINACION");
        } else
            entry.outError("Si el numerador es 0, toda la fraccion es "
                            +"igual a 0.", "FRACCION CON VALOR NULO");
    }
    return true;
}

void operacion(){
    String opciones[] = {"Sumar", "Restar", "Multiplicar", "Dividir",
                        "Exponente", "Negacion", "Comparar"};
    op = entry.entryOption("Seleccione la operacion", "OPERACIONES",
                           opciones);
    proceso(op);
}

void proceso(int seleccion){
    switch(seleccion){
        case 0 -> {data2(); resultado = r1.addition(r2);}
        case 1 -> {data2(); resultado = r1.substraccion(r2);}
        case 2 -> {data2(); resultado = r1.multiplicacion(r2);}
        case 3 -> {data2(); resultado = r1.division(r2);}
        case 4 -> { e = entry.entryInt(2,"Exponente",
                                        "Ingrese exponente");
                    resultado = r1.exponentiation(e);}
        case 5 -> resultado = r1.negation();
        case 6 -> {data2(); eq = r1.equals(r2);}
    }
}

```



```

    }
}

void resultados(){
    switch(op){
        case 0 -> {entry.outPlain(r1+" + "+r2+" = "+resultado,
                                "RESULTADO");}
        case 1 -> {entry.outPlain(r1+" - "+r2+" = "+resultado,
                                "RESULTADO");}
        case 2 -> {entry.outPlain(r1+" * "+r2+" = "+resultado,
                                "RESULTADO");}
        case 3 -> {entry.outPlain(r1+" / "+r2+" = "+resultado,
                                "RESULTADO");}
        case 4 -> {entry.outPlain(r1+"^"+e+" = "+resultado,
                                "RESULTADO");}
        case 5 -> {entry.outPlain("Negacion de "+r1+" = "
                                +resultado,"RESULTADO");}
        case 6 -> {entry.outPlain(r1+" == "+r2+" ? "+ eq,"COMPARACION");}
    }
}

void navegabilidad() {
    boolean continuar = true;
    String[] opciones = {"Sí", "No"};

    while (continuar) {
        data1();
        operacion();
        resultados();
        int reload = entry.entryOption("¿Desea realizar otra operación?",
                                      "Continuar", opciones);
        if (reload == 1) continuar = false;
    }
    entry.outInfo("Programa finalizado.", "Fin del programa");
}

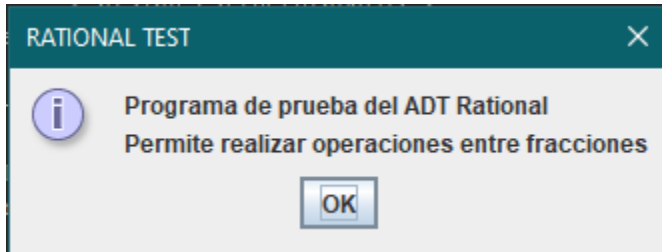
public static void main(String[] args) {
    TestRational test = new TestRational();

    test.meta();
    test.navegabilidad();
}
}

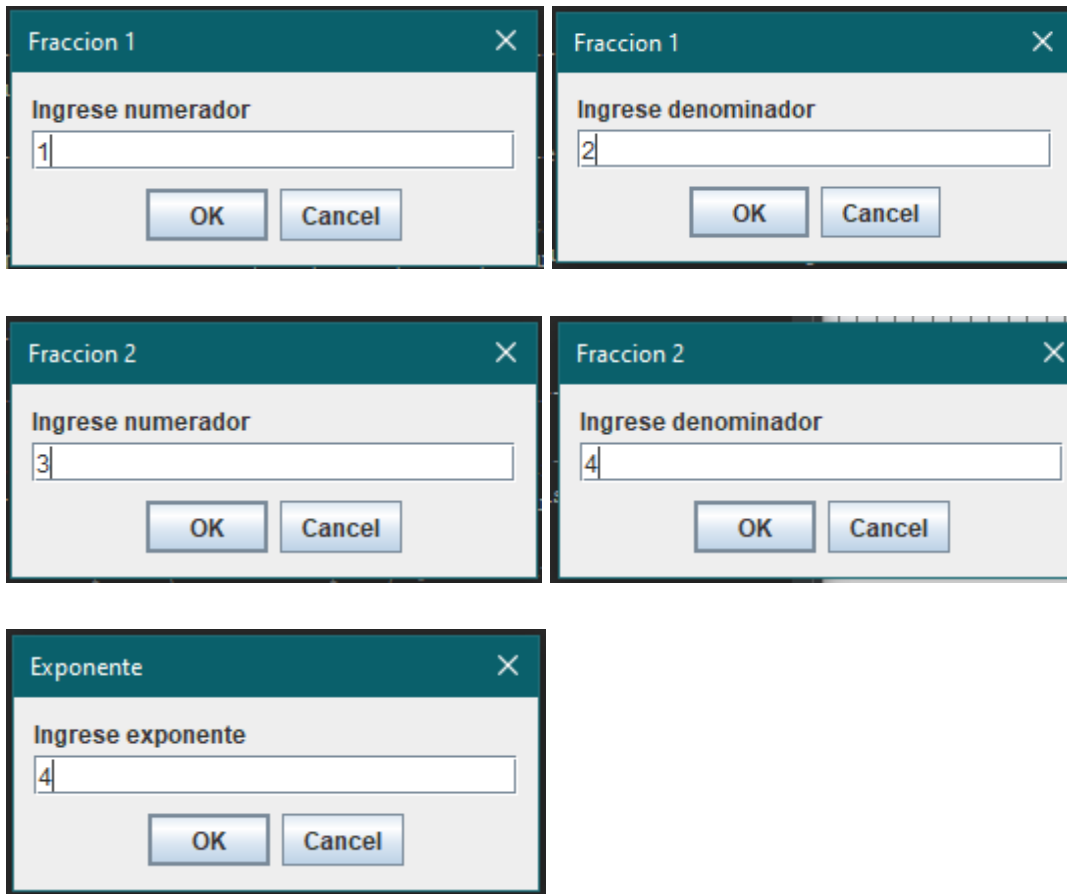
```

6. Resultados de corrida

Imagen demostrativa del Menú de inicio a continuación.



Imágenes demostrativas del programa solicitando el ingreso de datos al usuario a continuación.



Imágenes demostrativas del programa marcando errores en la validación de datos a continuación.

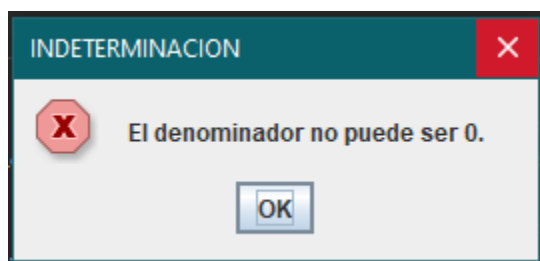
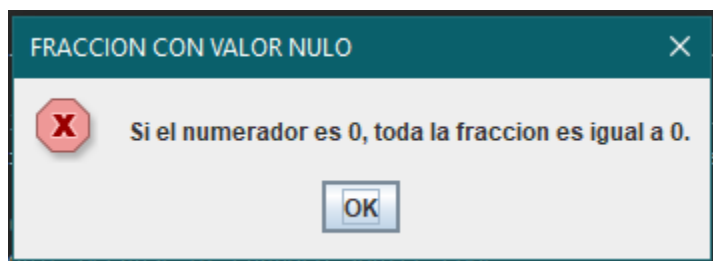
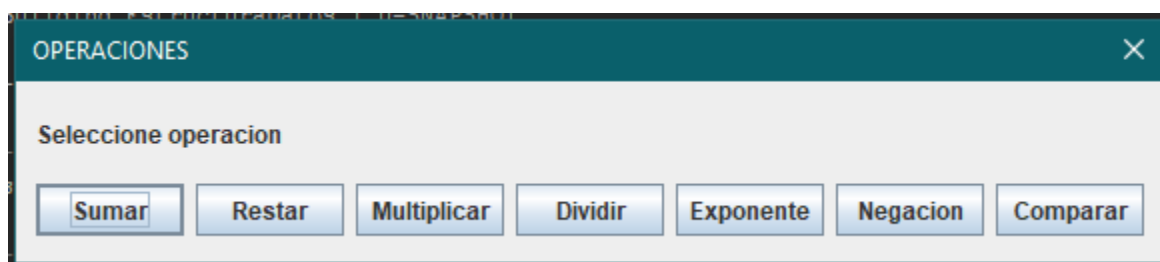


Imagen demostrativa del menú de opciones a continuación.



Imágenes demostrativas de los resultados de cada operacion del programa a continuación.

